

Implement 0/1 knapsack using Dynamic programming

Pseudocode

Algorithm KNAPSACK( $W$ ,  $wt[]$ ,  $val[]$ ,  $n$ )

Input:

$W \rightarrow$  Maximum capacity of knapsack,

$wt[] \rightarrow$  array of item weights.

$val[] \rightarrow$  array of item values.

$n \rightarrow$  number of items.

Output: Maximum profit that can be obtained.

create a table  $DP[0 \dots n][0 \dots W]$

for  $i \leftarrow 0$  to  $n$

$DP[i][0] \leftarrow 0$

end for

for  $w \leftarrow 0$  to  $W$

$DP[0][w] \leftarrow 0$

end for

for  $i \leftarrow 1$  to  $n$

for  $w \leftarrow 1$  to  $W$

If  $wt[i-1] \leq w$  Then

include  $\leftarrow val[i-1] + DP[i-1][w-wt[i-1]]$

exclude  $\leftarrow DP[i-1][w]$

$DP[i][w] \rightarrow \leftarrow \text{maximum}(\text{include}, \text{exclude})$

else

$DP[i][w] \leftarrow DP[i-1][w]$

end if

end for

end for

return  $DP[n][W]$

end

Code

```
#include <stdio.h>
```

```
int max(int a, int b)
```

```
{
```

```
    if (a > b)
```

```
        return a;
```

```
    else
```

```
        return b;
```

```
}
```

```
int knapsack(int W, int wt[], int val[], int n)
```

```
{
```

```
    int i, w;
```

```
    int K[n+1][W+1];
```

```
    for (i = 0; i <= n; i++)
```

```
    {
```

```
        for (w = 0; w <= W; w++)
```

```
        {
```

```
            if (i == 0 || w == 0)
```

```
            {
```



```

        K[i][w] = 0;
    }
    elseif (wt[i-1] <= w)
    {
        K[i][w] = max(
            val[i-1] + K[i-1][w-wt[i-1]],
            K[i-1][w]
        );
    }
    else
    {
        K[i][w] = K[i-1][w];
    }
}

return K[n][w];
}

```

```

int main()
{
    int n, w, i;
    printf("Enter number of items:");
    scanf("%d", &n);
    int wt[n], val[n];

    printf("Enter weights of items:\n");
    for (i=0; i<n; i++)
    {
        scanf("%d", &wt[i]);
    }
}

```

```
printf("Enter values of items:\n");
for (i=0; i<n; i++) {
    scanf("%d", &val[i]);
}

printf("Enter maximum capacity of knapsack:");
scanf("%d", &W);

int result = knapsack(W, wt, val, n);
printf("Maximum profit = %d\n", result);
return 0;
}
```

### output

Enter number of items: 5

Enter weights of items:

2 4 1 4 2

Enter values of items:

20 15 10 5 10

Enter maximum capacity of knapsack: 6

Maximum profit = 40

*[Signature]*  
11/5/26



494. Target Sum

```
int findTargetSumWays(int* nums, int numsSize, int target){
    int total = 0;
    for (int i = 0; i < numsSize; i++){
        total += nums[i];
    }
    if ((total + target) % 2 != 0 || abs(target) > total)
        return 0;
    int subsetSum = (total + target) / 2;
    int dp[subsetSum + 1];
    for (int i = 0; i <= subsetSum; i++)
        dp[i] = 0;
    dp[0] = 1;
    for (int i = 0; i < numsSize; i++){
        for (int j = subsetSum; j >= nums[i]; j--){
            dp[j] += dp[j - nums[i]];
        }
    }
    return dp[subsetSum];
}
```

~~Wrong~~  
11/5/20